

ー システムモデリングとシミュレーション ー オートマトン, セルオートマトン

工学部システム創成学科 (SDM) 藤井 秀樹

✉ fujii@sys.t.u-tokyo.ac.jp

2026年4月28日 システム工学基礎

講義（藤井担当分）の概要

□ 講義スケジュールと各回で扱うトピック

- ① 4月17日(金) 確率モデルの基礎
- ② 4月21日(火) 確率分布と数値計算
- ③ 4月24日(金) マルコフ連鎖, 待ち行列
- ④ 4月28日(火) オートマトン・セルオートマトン
- ⑤ 5月 1日(金) エージェント・マルチエージェント
(5月5日(火)は祝日)
- ⑥ 5月 8日(金) 自己組織化臨界
- ⑦ 5月12日(火) 連続体の離散モデル化

離散モデル (1)

□ 離散システム

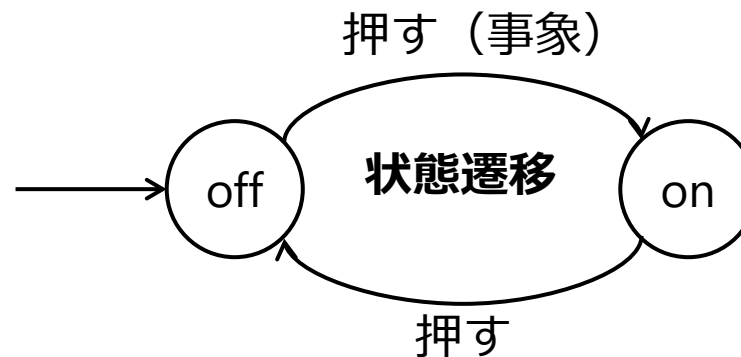
- 離散的な内部状態を持つ要素
または要素の集合

コンピュータを含むデジタルシステム
機械システムの基礎



自然現象の計算モデルとしても
応用可能

例) スイッチ

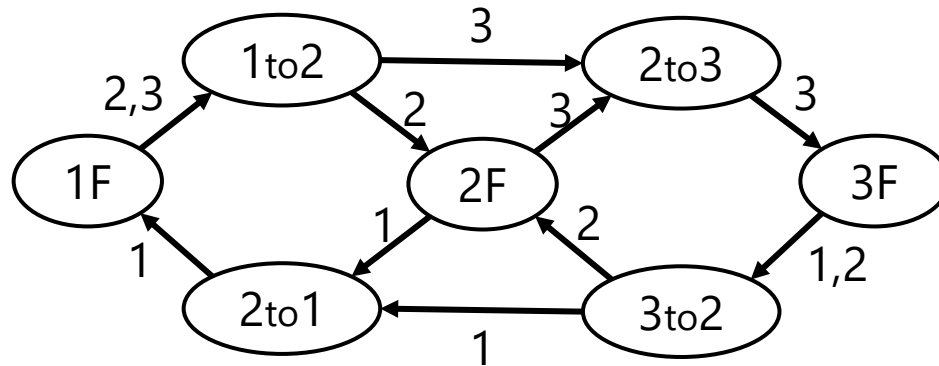


内部状態={off, on}

- 時間進行に伴って離散的な内部状態が遷移するシステム
→ マルコフ連鎖, 待ち行列モデルなど

離散モデル (2)

例) エレベータ



例) コンピュータ

- 内部状態：メインメモリの状態 = 膨大であるが有限個の01の列
- 状態遷移を生じさせる事象：入力, 出力, ある処理の開始・終了

例) 日本語・英語などの自然言語 → 言語処理, 言語解析

- (内部) 状態：有限個の文字の列

オートマトン

□ オートマトン automaton (=自動機械)

- 内部に状態を持ち, 状態遷移を行う要素
外部からの入力に対する応答

□ オートマトンの特徴

- 離散化された各時点において入力がある

「入力がある時点」で時間が離散化されている

- オートマトンへの入力 → **入力記号**

- 一般に, 個々のオートマトンが取り扱う入力記号の種類は有限
- 入力記号の有限集合 Σ (例: $\Sigma = \{a, b\}$, $\Sigma = \{1, 2, 3\}$)

- 状態遷移規則 (**状態遷移関数**) を持つ

オートマトンの定義 (1)

□ 定義（もう少し厳密に）

● 有限オートマトン finite automaton（有限状態オートマトンとも呼ぶ）

より厳密には決定性有限オートマトンと呼ぶ（後述）

- 取りうる状態の数が有限
- 5つの要素 $(Q, \Sigma, \delta, q_0, F)$ によって定義
 - Q : **状態**の種類（有限集合）
 - Σ : **入力記号**の種類（有限集合）
 - δ : **状態遷移関数**
 - » $Q \times \Sigma$ から Q への関数 $(Q \times \Sigma \rightarrow Q)$
 - $q_0 (\in Q)$: **初期状態** initial state (Q の元)
 - $F (\subseteq Q)$: **最終状態** final state (Q の部分集合)

オートマトンの定義 (2)

有限オートマトン $M = (Q, \Sigma, \delta, q_0, F)$ の例

$$Q = \{p, q\}, \Sigma = \{0, 1\}, q_0 = p, F = \{q\}$$

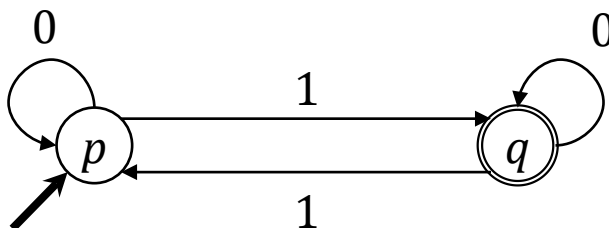
$$\delta(p, 0) = p, \delta(p, 1) = q, \delta(q, 0) = q, \delta(q, 1) = p$$

– 状態遷移表

状態	入力	
	0	1
$\rightarrow p$	p	q
$\odot q$	q	p

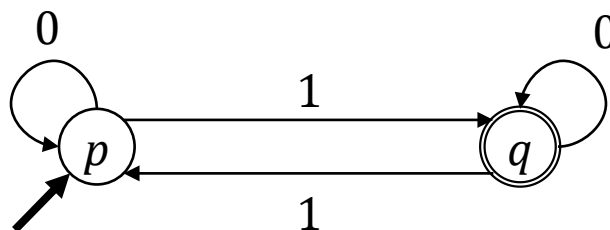
$\rightarrow$: 初期状態, $\odot$: 最終状態

– 状態遷移図



オートマトンの定義 (3)

□ オートマトンによる受理と非受理



● 受理

- 与えられた入力記号列を全部読みこんだ遷移先が最終状態
- 受理される記号列 : 1, 01, 111, 1011, ...

● 非受理

- 遷移先が最終状態でない or それ以上記号列を読み込めない (遷移先がない)
- 受理されない記号列 : 0, 11, 101, 1111, ...

上図では「0と1からなる記号列で, 1の個数が奇数であるもの」を受理

オートマトンの拡張 (1)

□ 非決定性有限オートマトン

- non-deterministic finite automaton (**NFA**)
 - これまでの説明は**決定性有限オートマトン** (deterministic finite automaton, **DFA**)
- 5つの要素 ($Q, \Sigma, \delta, q_0, F$) によって定義
 - Q : 状態の有限集合, Σ : 入力記号の有限集合, q_0 : 初期状態, F : 最終状態
 - δ : 状態遷移関数
 - $Q \times \Sigma$ から 2^Q への関数 ($Q \times \Sigma \rightarrow 2^Q$) ... 遷移先が1つでなく集合

べき集合 2^X

集合 X の部分集合すべてからなる集合 (集合族 : 集合の集合)

例) $2^{\{a,b\}} = \{\emptyset, \{a\}, \{b\}, \{a,b\}\}$

オートマトンの拡張 (2)

非決定性有限オートマトン $M = (Q, \Sigma, \delta, q_0, F)$ の例

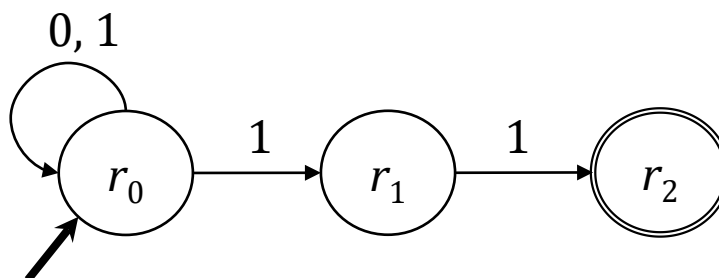
$$Q = \{r_0, r_1, r_2\}, \quad \Sigma = \{0, 1\}, \quad q_0 = r_0, \quad F = \{r_2\},$$
$$\delta(r_0, 0) = \{r_0\}, \quad \delta(r_0, 1) = \{r_0, r_1\}, \quad \delta(r_1, 0) = \emptyset,$$
$$\delta(r_1, 1) = \{r_2\}, \quad \delta(r_2, 0) = \emptyset, \quad \delta(r_2, 1) = \emptyset$$

$\emptyset$: 遷移しない

状態遷移表

状態	入力	
	0	1
$\rightarrow r_0$	$\{r_0\}$	$\{r_0, r_1\}$
r_1	$\emptyset$	$\{r_2\}$
$\odot r_2$	$\emptyset$	$\emptyset$

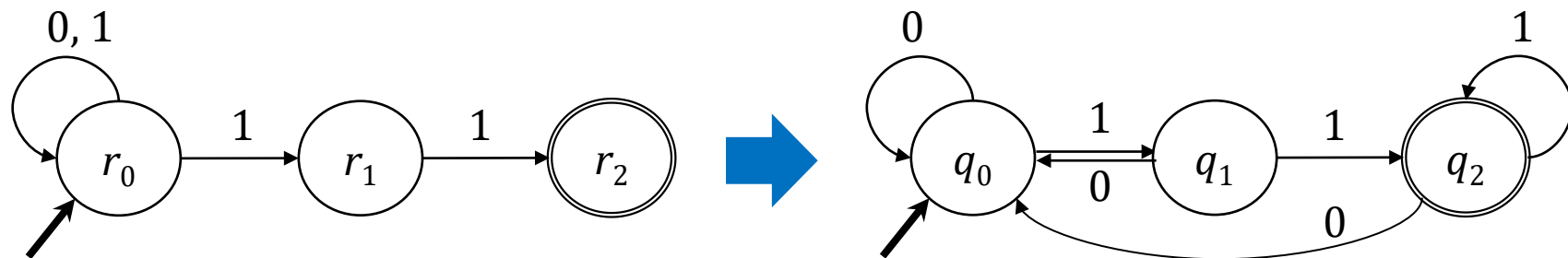
状態遷移図



オートマトンの拡張 (3)

□ NFAからDFAへ

- NFAの状態数を n とすると、遷移中に取りうる到達可能な状態集合の数はたかだか 2^n (べき集合の元の個数)
- たかだか 2^n 個の状態集合のそれぞれを新しく1つの状態とみなし、それらの遷移を記述することで、NFAと等価なDFAを得られる。

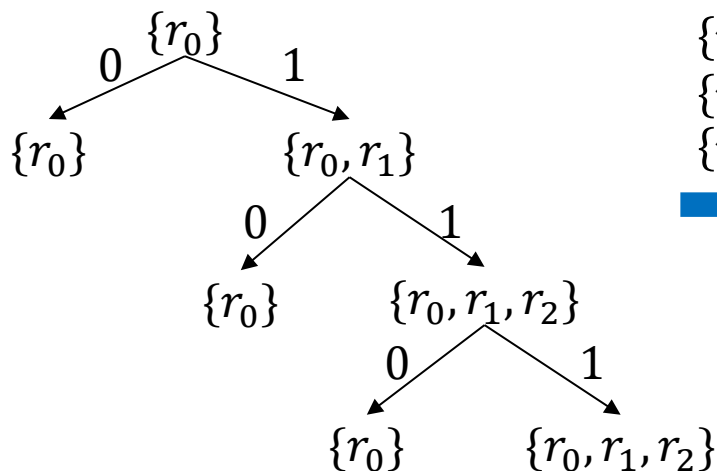
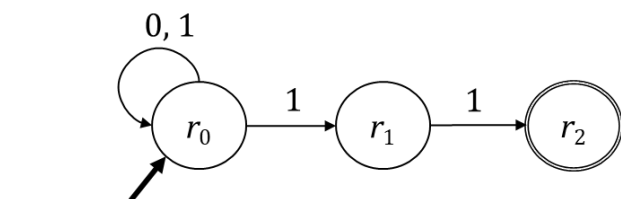


オートマトンの拡張 (4)

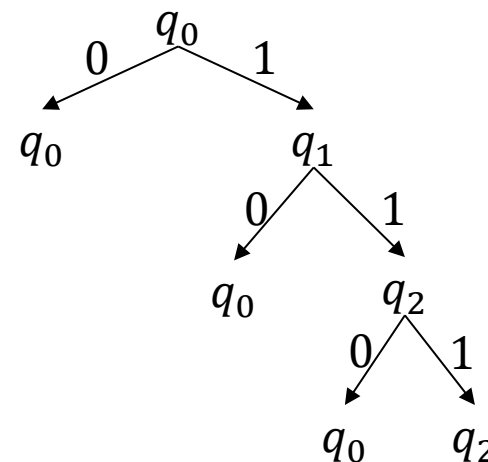
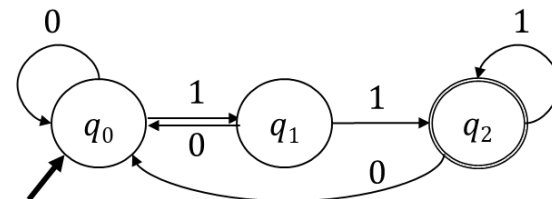
□ NFAからDFAへの変換方法

● 部分集合構成法 subset construction

- NFAの遷移状況をツリー構造で表現する
 - 既に出現した状態集合はそれ以上展開しない
- 出現する状態集合を新たな状態で置き換える



$\{r_0\} = q_0,$
 $\{r_0, r_1\} = q_1,$
 $\{r_0, r_1, r_2\} = q_2$ とする



言語とオートマトン (1)

□ 形式言語

- 言語を抽象化したもの

□ 言語 = 文字列の集合

- Σ : 文字種の集合 (**アルファベット**・字母)
- Σ^* : Σ に属する文字を有限個並べてできる文字列の集合
 - 「0個並べてできる文字列」も含む → **空文字列** (ヌル文字列) ε
- 言語とは Σ^* の部分集合と定義できる

例1) $\Sigma = \{0,1\}$ であるとき,

$$\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, \dots\}$$

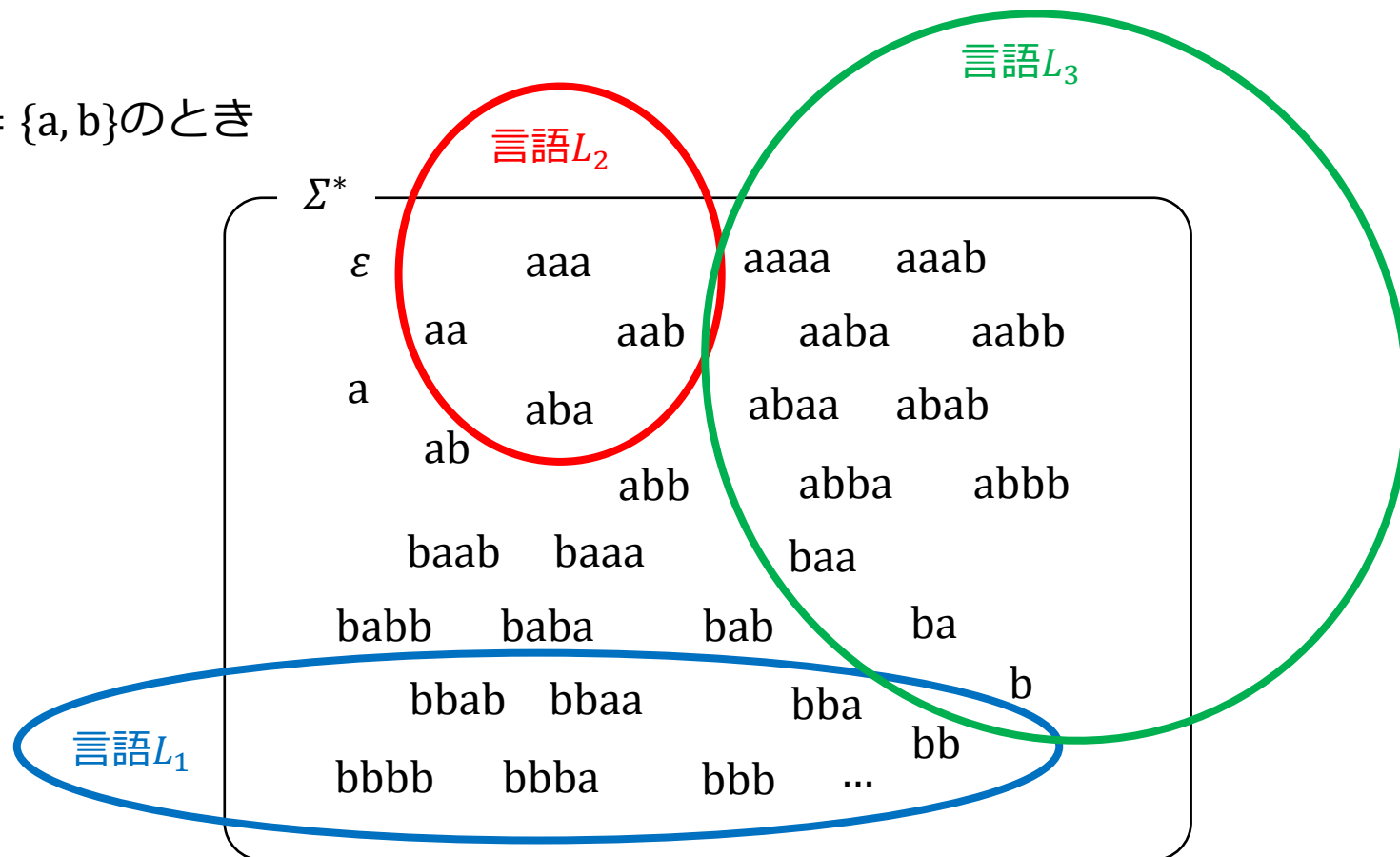
例2) $\Sigma = \{a, b, \dots, z, A, B, \dots, Z\}$ であるとき,

英単語の集合 (言語) は $\Sigma^* = \{a, b, \dots, z, A, B, \dots, Z\}^*$ の部分集合

言語とオートマトン (2)

- Σ : 文字種の集合 (アルファベット)
- Σ^* : Σ 上の文字列全体の集合
- 言語: Σ^* の部分集合

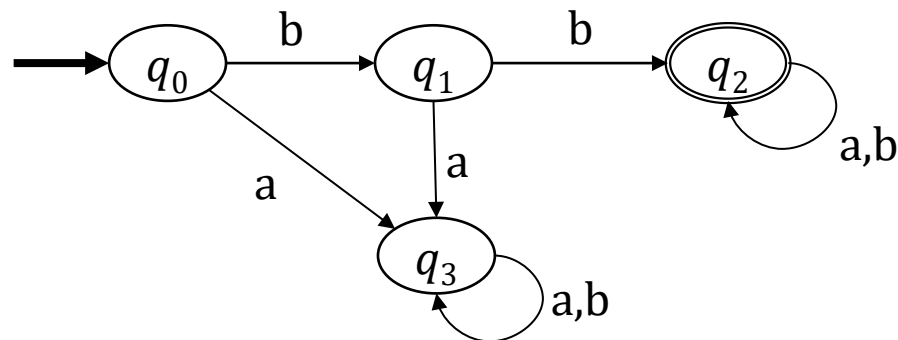
例) $\Sigma = \{a, b\}$ のとき



言語とオートマトン (3)

□ オートマトンが受理する言語 (←記号列の集合)

例)



このオートマトンが受理する言語：

$\{bbx \mid x \in \{a, b\}^*\}$ (aとbから構成されbbで始まる文字列の集合)

「オートマトンが記号列を受理する」 = 与えられた入力記号列を全部読みこんだ遷移先が最終状態

● オートマトンによる言語の表現

– 文字の有限集合 Σ 上の言語

→ ちょうどそれだけを受理するオートマトンを定義可能

オートマトンが受理する言語 (1)

□ オートマトンによる受理 (別の定義)

- 関数 $\hat{\delta}: Q \times \Sigma^* \rightarrow Q$ を次のように定義する

遷移関数は $\delta: Q \times \Sigma \rightarrow Q$

① $\hat{\delta}(q, \varepsilon) = q, \forall q \in Q$

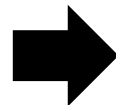
② $\hat{\delta}(q, a) = \delta(q, a), \forall q \in Q, \forall a \in \Sigma$

③ $\hat{\delta}(q, w) = \delta(\hat{\delta}(q, w'), a), \forall w = w'a \in \Sigma^+$

※ $\Sigma^+ = \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots$
(正閉包: 空文字を含めない)

以下の状態遷移表を持つDFAに
言語 "1011" を与える

状態	入力	
	0	1
$\rightarrow p$	p	q
$\odot q$	q	p



$$\begin{aligned}\hat{\delta}(p, 1011) &= \delta(\hat{\delta}(p, 101), 1) \\ &= \delta(\delta(\hat{\delta}(p, 10), 1), 1) \\ &= \delta(\delta(\delta(\hat{\delta}(p, 1), 0), 1), 1) \\ &= \delta(\delta(\delta(\delta(p, 1), 0), 1), 1) \\ &= \delta(\delta(\delta(q, 0), 1), 1) \\ &= \delta(\delta(q, 1), 1) \\ &= \delta(p, 1) = q\end{aligned}$$

オートマトンが受理する言語 (2)

□ 決定性有限オートマトンが受理する言語

- DFA: $M = (Q, \Sigma, \delta, q_0, F)$ によって受理される言語 $L(M)$

$$L(M) = \{w \mid \hat{\delta}(q_0, w) \in F\}$$

□ 非決定性有限オートマトンが受理する言語

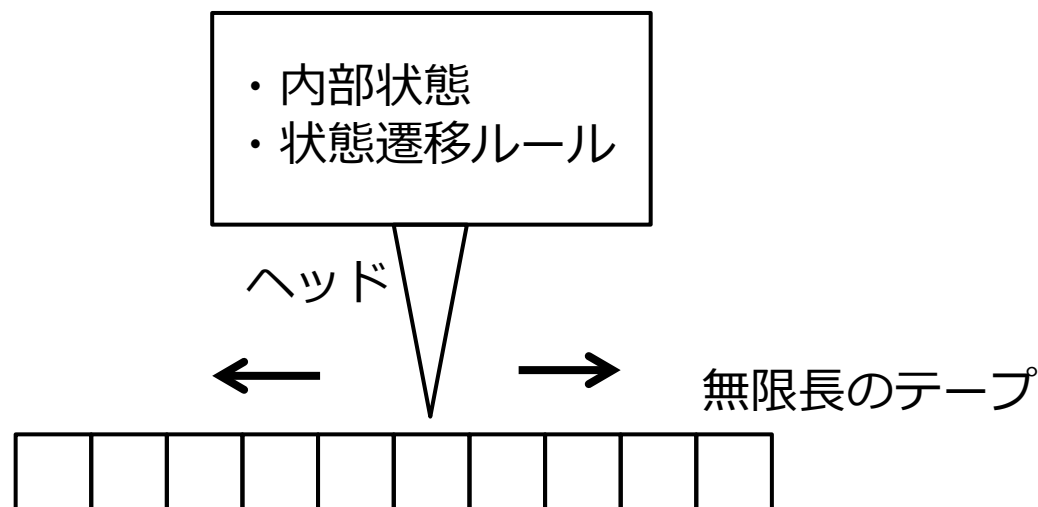
- NFA: $M' = (Q', \Sigma', \hat{\delta}', q'_0, F')$ によって受理される言語 $L(M')$

$$L(M') = \{w \mid \hat{\delta}'(q'_0, w) \cap F' \neq \emptyset\}$$

チューリングマシン (1)

□ チューリングマシン Turing machine

- Alan Turingが1936年に提唱
- オートマトン+テープへの読み書き（外部記憶）
 - オートマトンは内部状態しか持たないため記憶ができない
- 理想化された仮想機械
 - コンピュータ上で実行可能なアルゴリズムは全てチューリングマシン上で実行可能



チューリングマシン (2)

□ チューリングマシンの定義

- 7つの要素 ($Q, \Gamma, b, \Sigma, \delta, q_0, F$) によって定義
 - Q : 状態の有限集合, Σ : 入力記号の有限集合, q_0 : 初期状態, F : 最終状態
 - Γ : 使用される記号の集合 ($\Gamma = \Sigma \cup \{b\}$)
 - b : 空白記号
 - δ : **状態遷移関数**
 - » $Q \times \Sigma$ から $Q \times \Gamma \times \{L, R\}$ への関数 ($Q \times \Sigma \rightarrow Q \times \Gamma \times \{L, R\}$)

- 動作表

		内部状態	
		q_0	q_1
入力記号	a_1	$a_1 R q_0$	$a_1 R q_0$
	a_2	$a_1 \uparrow$	$a_2 L q_1$

最終状態

$\Lambda R q_0$

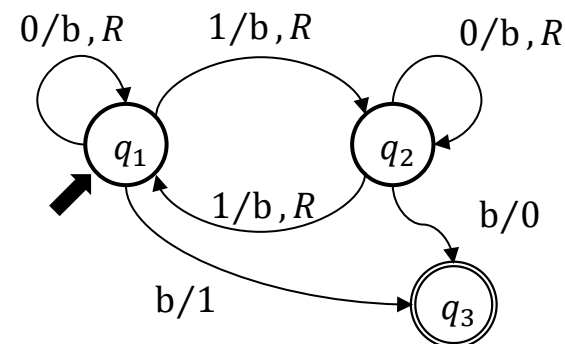
- ・ テープに書き込む記号
- ・ ヘッドの動く向き
- ・ 次の内部状態

チューリングマシン (3)

チューリングマシン $M = (Q, \Gamma, b, \Sigma, \delta, q_0, F)$ の例

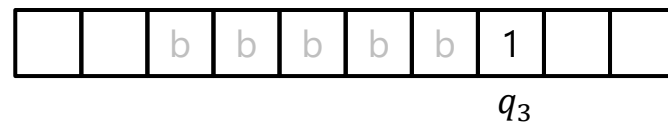
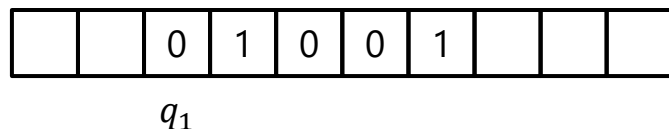
$$Q = \{q_1, q_2, q_3\}, \quad \Gamma = \{b, 0, 1\}, \quad \Sigma = \{0, 1\}, \quad q_0 = q_1, \quad F = \{q_3\}$$

δ	b	0	1
$\rightarrow q_1$	$1q_3$	bRq_1	bRq_2
q_2	$0q_3$	bRq_2	bRq_1



このチューリングマシンは,

テープに書かれた0と1を消去しながら (空白文字にしながら) 1の数を調べ,
偶数個の1がテープ上に書かれていれば「1」, そうでなければ「0」を出力して停止する



bは実際には空白

チューリングマシンと計算

□ 計算機とチューリングマシン

- 計算機で原理的に解ける問題 = チューリングマシンで解ける問題

□ 万能チューリングマシン

- あらゆるチューリングマシンを模倣できる, あるチューリングマシン
 - 万能チューリングマシンの入力は, 模倣対象のチューリングマシンを記述する記号列と, 模倣対象のチューリングマシンへの入力記号列

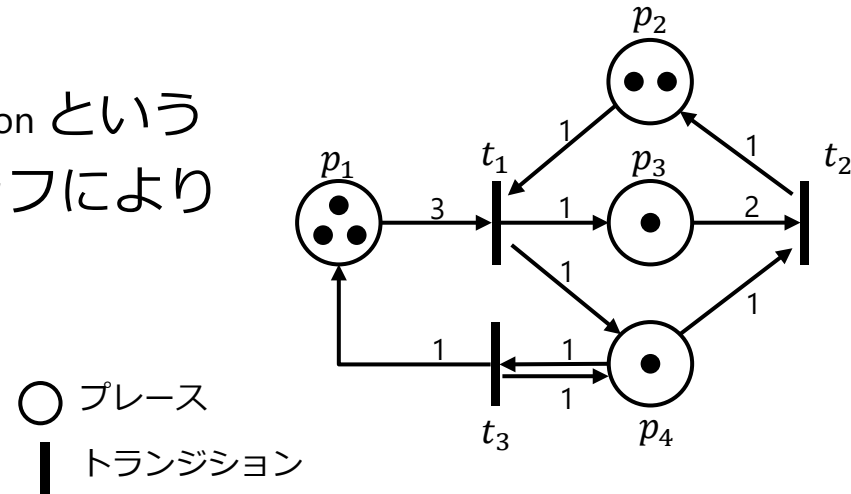
□ チューリング完全

- 万能チューリングマシンの持つ性質を再現できる計算メカニズム
 - チューリング完全であることを示すには, チューリング完全であると既に分かっている別の計算メカニズムを模倣できることを示せばよい
- プログラミング言語の必要条件

ペトリネット (1)

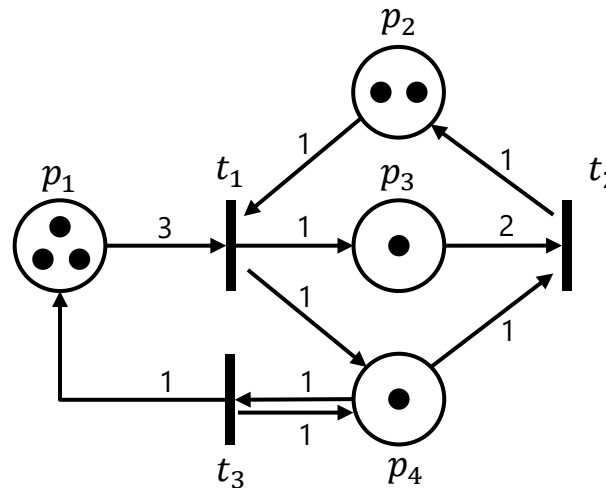
□ ペトリネット Petri net

- プレース place, トランジション transition という2種類のノードを持った二部有向グラフによりシステムの状態遷移を表現する



- ペトリネット $N(P, T, F, W, M_0)$
 - $P = \{p_1, p_2, p_3, p_4\}$: プレースの有限集合
 - $T = \{t_1, t_2, t_3\}$: トランジションの有限集合
 - $F = \{(p_1, t_1), (t_1, p_3), \dots\} \subseteq (P \times T) \cup (T \times P)$: アークの集合
 - $W: F \mapsto \{1, 2, \dots\}$: アークの重み
 - $m_0 = [3 \ 2 \ 1 \ 1]^T$: 初期マーキング

ペトリネット (2)

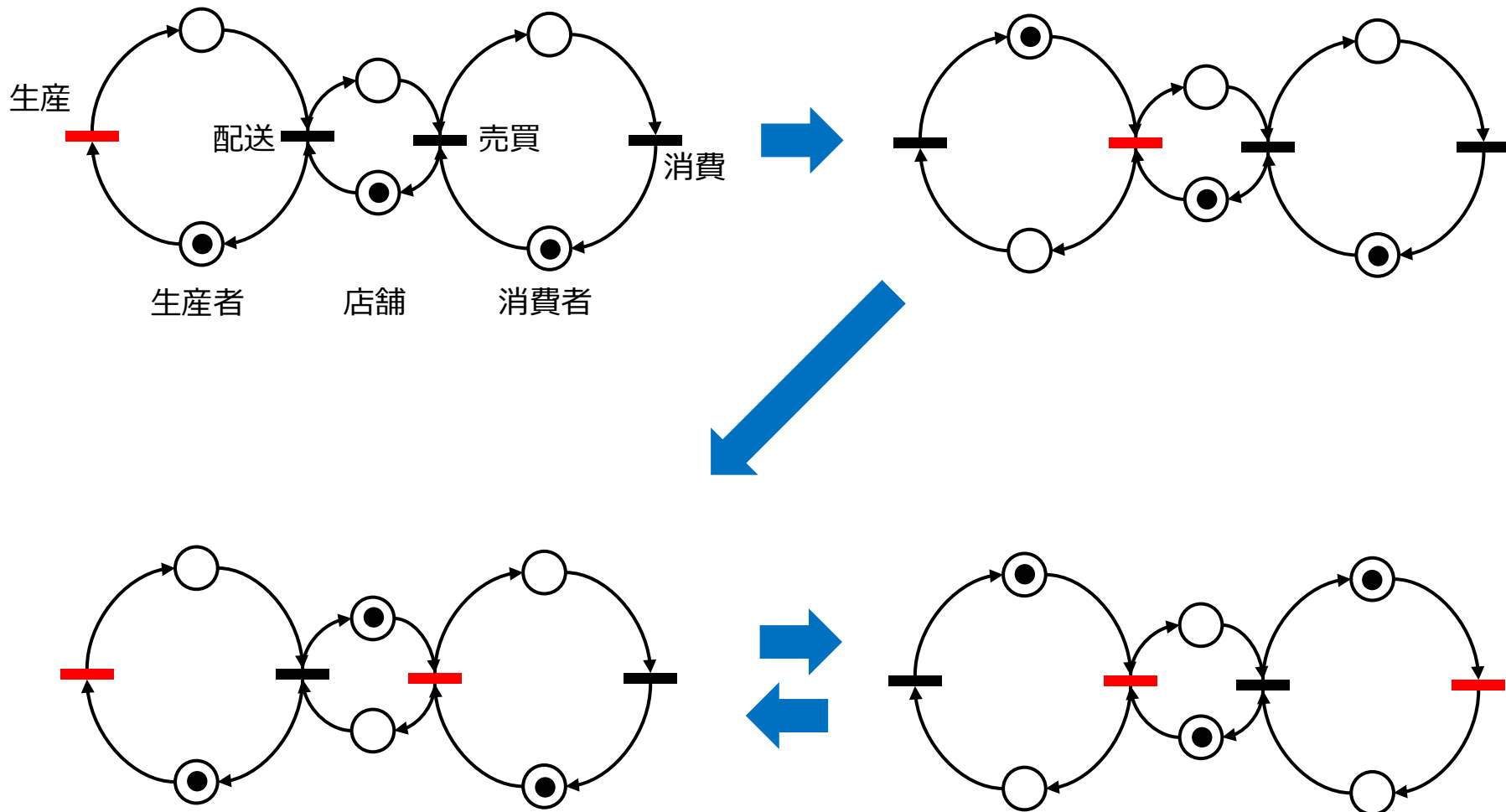


○ プレース
| トランジション

- プレース
 - 条件をあらわす. 非負整数個のトークン (●) が置かれる.
- トランジション
 - 事象をあらわす.
 - トランジションのすべての入力プレースが入力アークの重み以上の個数のトークンを持つとき, t は**発火** firing する.
 - 各入力プレースから入力アークの重みの数だけトークンが取り除かれる
 - 各出力プレースに出力アークの重みの数だけトークンが追加される

ペトリネット (3)

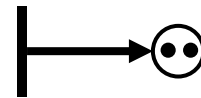
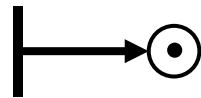
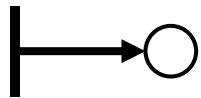
例) 生産者・消費者のモデル



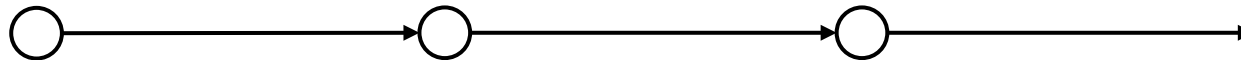
ペトリネットとオートマトン (1)

□ オートマトンとの関係

- ペトリネットは拡張されたオートマトンの一種
 - オートマトンは状態遷移図のノードそのものが状態を表現するのに対し、ペトリネットはプレース（ノード）に置かれたトークンで状態を表現
 - トークン数の変更に対して柔軟
- ペトリネットは有限オートマトンより柔軟性が高い
 - 無限状態の表現



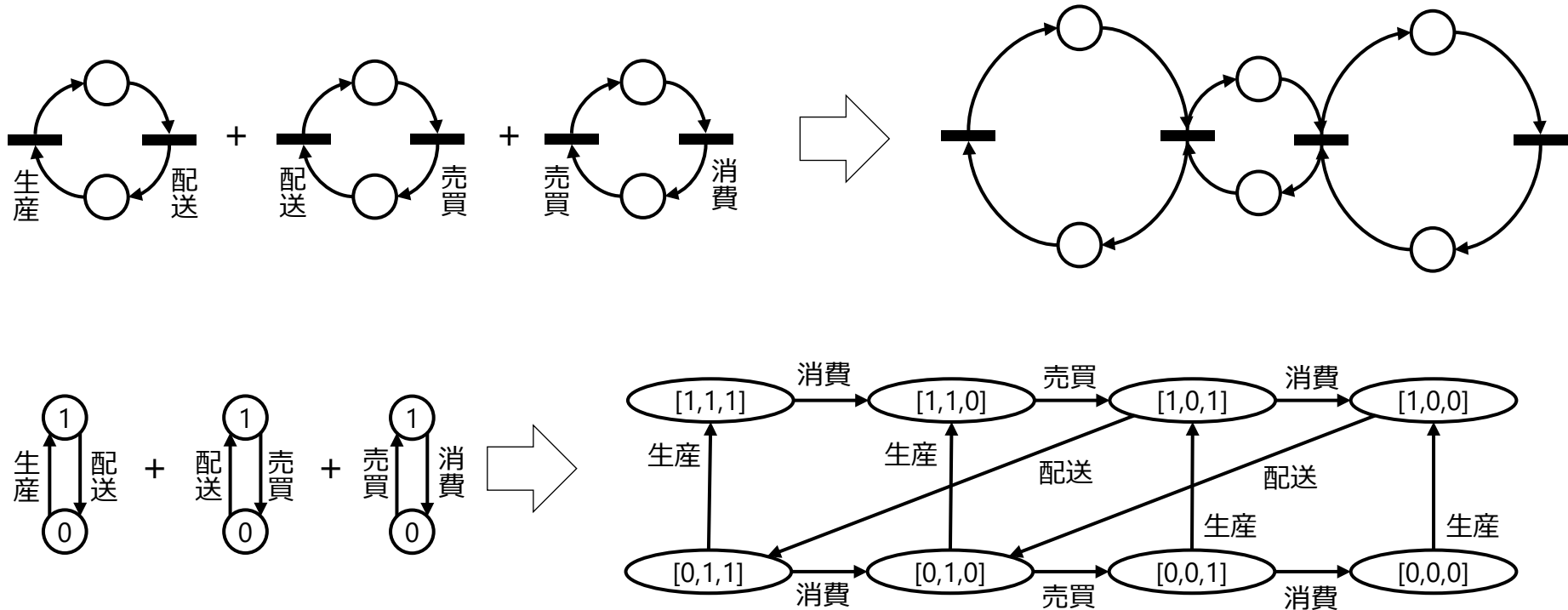
トークンが増える



ノードが増える

ペトリネットとオートマトン (2)

- ペトリネットは有限オートマトンより柔軟性が高い
 - 合成の容易さ



ペトリネットは全体のノード数が各コンポーネントのノード数の和になるが
有限オートマトンは各コンポーネントのノード数の積になる

ペトリネットとオートマトン (3)

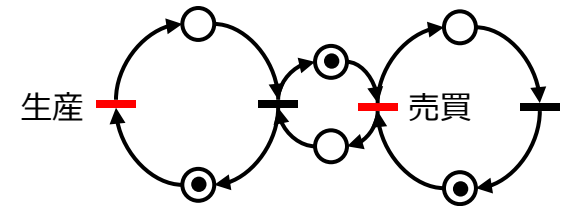
- ペトリネットは有限オートマトンより柔軟性が高い

- 並行処理の表現

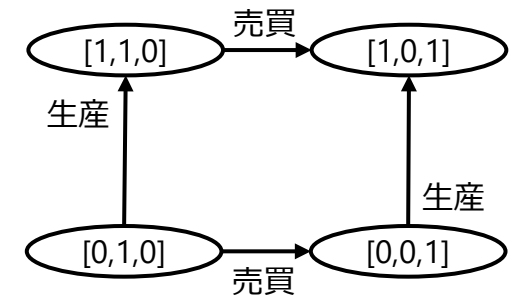
- 右の例で，生産と売買は並行処理可能
- 並行処理可能 = 独立 = 競合しない

» 競合しない条件：

2つのトランジションが共通の入力スペースを持たない，
あるいは，持ったとしても両方のトランジションの発火に必要な
十分の数のトークンが存在する



- オートマトンでは生産→売買，売買→生産の
どちらも起こりうる



セルオートマトン

□ セルオートマトン(CA) cellular automata の概念

- 有限オートマトンを格子（グリッド）状に多数配置したもの → セル
- 空間が同じ大きさのセルに区切られている
 - 1次元でも2次元でも3次元でもよい
- 各セルは有限個の状態のうち1つをとる
- 次の時刻のセルの状態は自身および近傍セルの現在の状態に依存する

1次元セルオートマトン

□ 1次元CA

...	a_t^{i-2}	a_t^{i-1}	a_t^i	a_t^{i+1}	a_t^{i+2}	...
-----	-------------	-------------	---------	-------------	-------------	-----

- a_t^i : 時刻 t におけるセル i の状態 (状態数 k)
- 状態遷移関数

$$a_{t+1}^i = f(a_t^{i-r}, a_t^{i-r+1}, \dots, a_t^{i-1}, a_t^i, a_t^{i+1}, \dots, a_t^{i+r-1}, a_t^{i+r})$$

- 次の時刻 $t + 1$ のセル i の状態は, 近傍セルの状態として a_t^{i-r} から a_t^{i+r} までの $2r + 1$ 個を考慮し, 近傍セルの状態を変数としてとる関数 f によって定まる.
- $k = 2, r = 1 \rightarrow$ **2状態3近傍の1次元セルオートマトン**
 - elementary cellular automata, ECAとも呼ばれる

ECA (1)

□ 2状態3近傍の1次元CA

...	a_t^{i-1}	a_t^i	a_t^{i+1}	...
-----	-------------	---------	-------------	-----

● Wolframコード (Wolframのルール)

$$(f\text{のWolframコード})_{10} = (f(1,1,1)f(1,1,0) \dots f(0,0,1)f(0,0,0))_2$$

時刻 t での状態 (左隣, 自分, 右隣)	111	110	101	100	011	010	001	000
時刻 $t + 1$ での状態 (自分)	s_7	s_6	s_5	s_4	s_3	s_2	s_1	s_0

$$(f\text{のWolframコード})_{10} = \sum_{i=0}^7 s_i \cdot 2^i$$

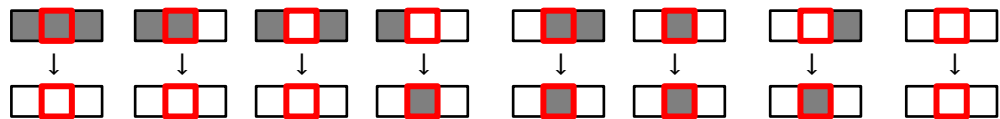
ECA (2)

例) ルール30

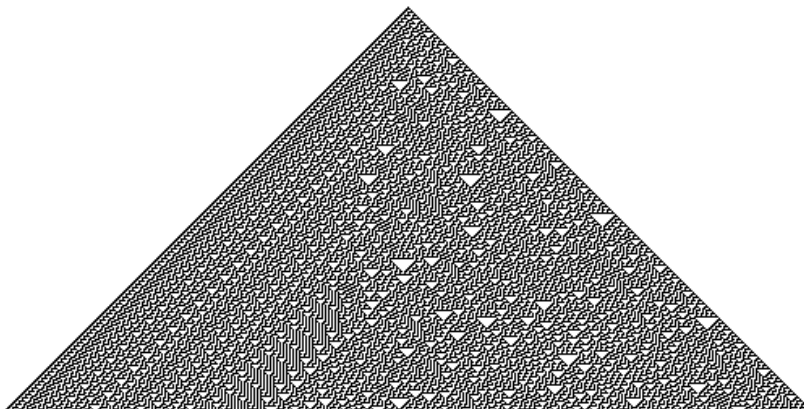
- セルの状態は2値 (0:□, 1:■)

ルール30 ($30_{10} = 00011110_2$)

時刻 t での状態 (左隣, 自分, 右隣)	111	110	101	100	011	010	001	000
時刻 $t+1$ での状態 (自分)	0	0	0	1	1	1	1	0



時間の進展
↓

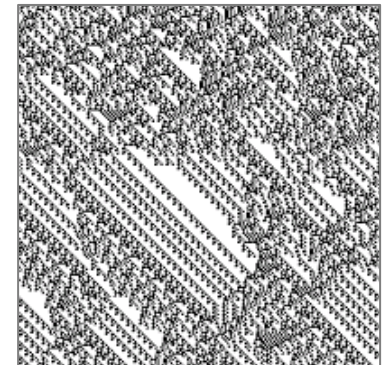
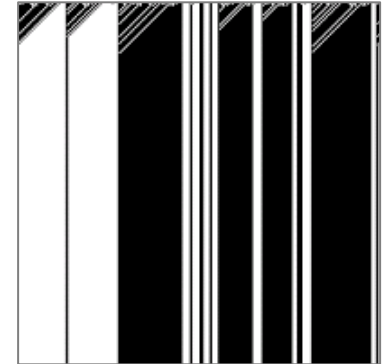
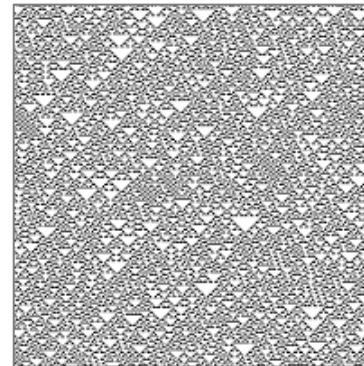
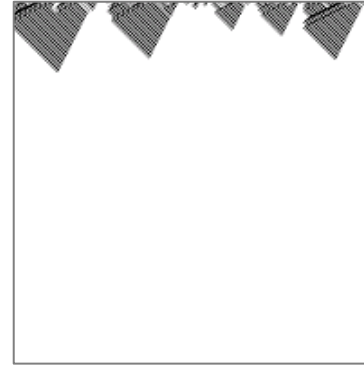


Coombes, S., "The geometry and pigmentation of seashells",
Techn. Ber. Department of Mathematical Sciences (2009) より引用

1次元CAの時空間パターン

□ 時間発展により得られるパターンの分類

- クラス 1
 - 全てのセルの状態が均一
- クラス 2
 - 周期的に繰り返すパターン
- クラス 3
 - 非周期的でカオス的な振る舞い
- クラス 4
 - 予測しがたい非常に複雑なパターン



その他の1次元セルオートマトン

□ 2状態5近傍CA

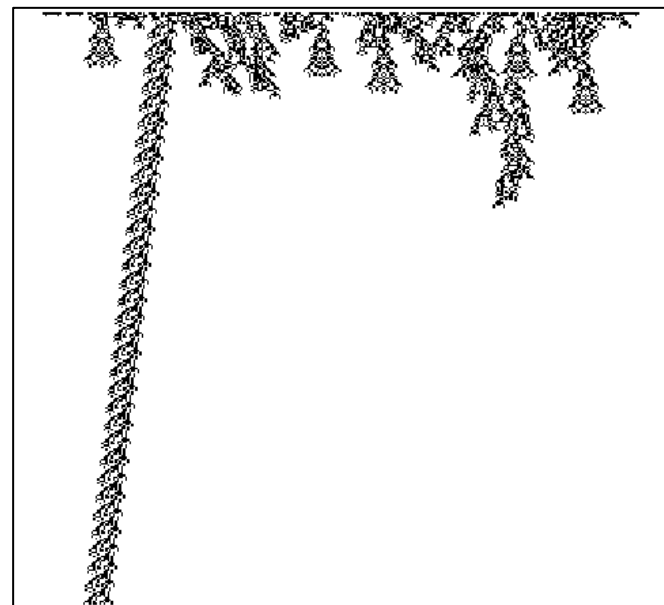
● 総和型ルール

- 時刻 $t + 1$ におけるセルの値は、時刻 t における近傍のセル群の値の総和だけに依存して決定される

総和型ルール20 ($20_{10} = 010100_2$)

時刻 t での状態例	11111	11101	10011	10001	01000	00000
和	5	4	3	2	1	0
時刻 $t+1$ での状態 (中央セル)	0	1	0	1	0	0

2状態5近傍で総和型ルールを使わない（ECAと同様にルールを定義する）と、ルールの総数は4,294,967,296 (約43億)



セルオートマトンの応用

□ 2次元CA

- 伝播モデル
- マジヨリティモデル
- ライフゲーム

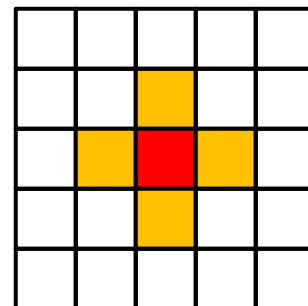
□ セルオートマトンの応用

- 要素同士の単純な相互作用から全体の複雑なダイナミクスを創発
- 隣接の概念の緩和 → ネットワークモデル
- セルの自律的な相互作用 → マルチエージェントシステム

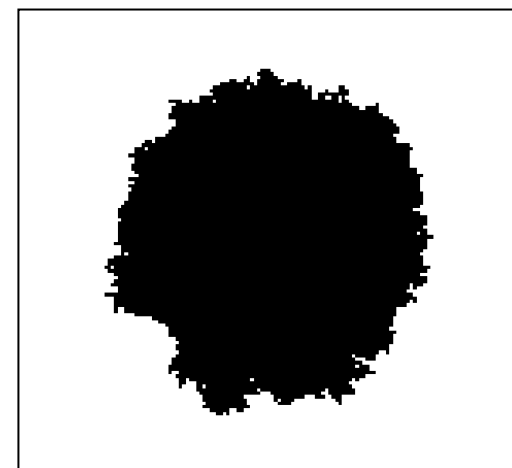
2次元CAの動作例 (1)

□ 伝播モデル

- セルの状態は2値 (□: 0, ■: 1)
- 状態遷移規則
 - 白いセル
 - Neumann近傍の隣接4セルに黒いセルがあれば、ある確率で自身の状態を白から黒に変える
 - 近傍に黒いセルが複数ある場合はその数だけ繰り返す
 - 黒いセル
 - 自身の状態を保つ



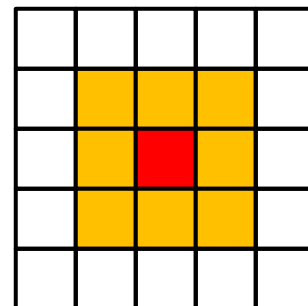
近傍セル (Neumann近傍)



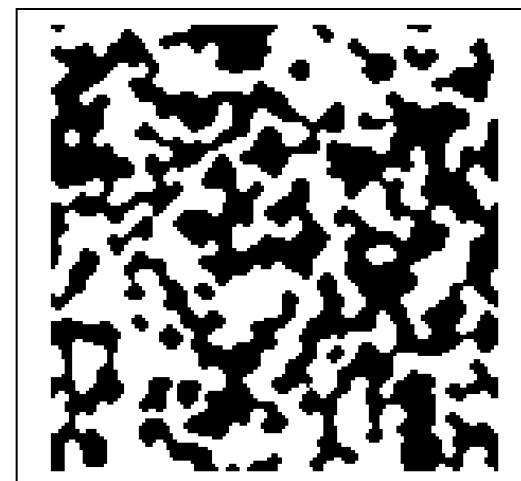
2次元CAの動作例 (2)

□ マジョリティモデル

- セルの状態は2値 (□: 0, ■: 1)
- 状態遷移規則
 - Moore近傍8セル中の白と黒を数え, 数が多い方に自身の色を合わせる
 - 白と黒が同数 (4個ずつ) の場合は自身の状態を保つ



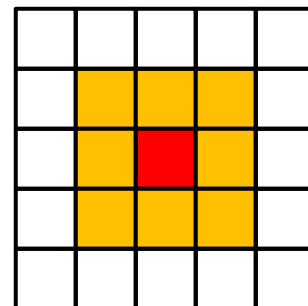
近傍セル (Moore近傍)



2次元CAの動作例 (3)

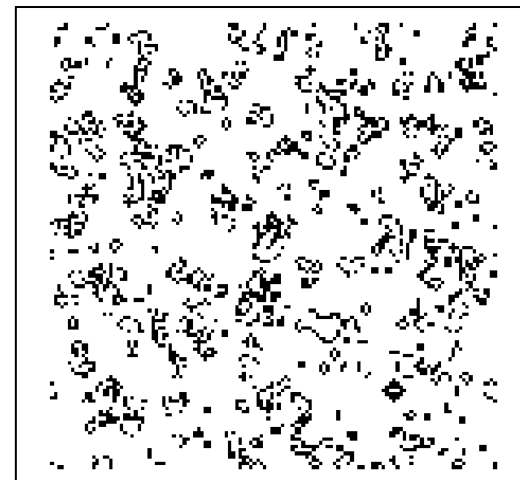
□ ライフゲーム

- セルの状態は2値
(□: 0 (死んでいる) , ■: 1 (生きている))



近傍セル (Moore近傍)

- 状態遷移規則
 - 生きているセルは, Moore近傍に2 or 3つのセルが生きていれば生き残る.
 - 過疎でも過密でも生き抜くことはできない.
 - 死んでいるセルでは, 近傍に3つのセルが生きているときに生命が生まれる.



本日の学習内容

□ 有限オートマトン

- 状態, 入力記号, 初期状態, 最終状態, 状態遷移関数によって定義
- 言語の受理（認識・選別）に応用される
- + 外部記憶 → チューリングマシン

□ セルオートマトン

- 有限オートマトンを格子（グリッド）状に多数配置したもの
- 自身および近傍セルの現在の状態に依存
- もっとも単純なのは2状態3近傍の1次元セルオートマトン (ECA)
 - 遷移規則は256通り → 0から255に対応付けられる（Wolframコード）
- 物理現象・社会現象のシンプルなモデル化に応用される